

JavaScript Interview Questions

I. JavaScript Basics & Fundamentals (1-20)

1. **Q: What is JavaScript? A:** JavaScript is a high-level, interpreted (or JIT-compiled) programming language primarily known as the scripting language for Web pages. It is also used in many non-browser environments thanks to Node.js. It's prototype-based, multi-paradigm, single-threaded, dynamic, and supports object-oriented, imperative, and declarative (e.g., functional programming) styles.
2. **Q: What are the different data types in JavaScript? A:** JavaScript has two main categories of data types:
 - **Primitive Types:**
 - **string:** Sequence of characters.
 - **number:** Numeric values (both integer and floating-point).
 - **bigint:** For arbitrarily large integers.
 - **boolean:** `true` or `false`.
 - **undefined:** A variable that has been declared but not assigned a value.
 - **symbol:** Unique and immutable primitive value.
 - **null:** Represents the intentional absence of any object value.
 - **Reference Type (Object):**
 - **object:** Collections of properties (including `Array`, `Function`, `Date`, `RegExp`, etc.).
3. **Q: What is the difference between `null` and `undefined`? A:**
 - **undefined** typically means a variable has been declared but not yet assigned a value. It's also the default return value of functions that don't explicitly return anything.
 - **null** is an assignment value. It can be assigned to a variable as a representation of "no value" or an intentional absence of any object value.
4. **Q: What is the difference between `==` and `===`? A:**
 - **==** (Abstract Equality Comparison): Compares two values for equality *after* performing type coercion if necessary.
 - **===** (Strict Equality Comparison): Compares two values for equality *without* performing type coercion. Both value and type must be the same. It's generally recommended to use `===`.

```
console.log(5 == "5"); // true (coercion happens)
console.log(5 === "5"); // false (types are different)
```

5. **Q: What is type coercion in JavaScript? A:** Type coercion is the automatic or implicit conversion of values from one data type to another (e.g., string to number, number to boolean). This happens when operators

are applied to values of different types.

6. **Q: What are `var`, `let`, and `const`? What are their differences?**

A:

- **`var`:**
 - Function-scoped or globally-scoped.
 - Can be re-declared and updated.
 - Hoisted to the top of its scope and initialized with **`undefined`**.
- **`let`:**
 - Block-scoped (`{}`).
 - Can be updated but not re-declared within the same scope.
 - Hoisted but not initialized (Temporal Dead Zone).
- **`const`:**
 - Block-scoped.
 - Cannot be updated or re-declared.
 - Must be initialized during declaration.
 - Hoisted but not initialized (Temporal Dead Zone).
 - For objects and arrays, the reference is constant, but the content can be modified.

7. **Q: What is “hoisting” in JavaScript? A:** Hoisting is JavaScript’s default behavior of moving declarations to the top of their current scope (script or function scope for **`var`** and function declarations, block scope for **`let`** and **`const`** though they are not initialized).

- Function declarations are fully hoisted (name and body).
- **`var`** declarations are hoisted and initialized with **`undefined`**.
- **`let`** and **`const`** declarations are hoisted but not initialized, leading to a Temporal Dead Zone until their declaration is encountered.

8. **Q: What is the Temporal Dead Zone (TDZ)? A:** The Temporal Dead Zone is the period between entering scope and a **`let`** or **`const`** variable’s declaration. Accessing the variable in the TDZ results in a **`ReferenceError`**.

9. **Q: What are global variables? How can they be problematic? A:**

Global variables are variables declared outside any function or block, or variables assigned a value without prior declaration (in non-strict mode, this creates a global property). They are accessible from any part of the code. Problems:

- **Name Collisions:** Different parts of the code might unknowingly use the same global variable name, leading to bugs.
- **Reduced Readability/Maintainability:** It’s harder to track where a global variable is modified.
- **Testing:** Makes unit testing harder as global state can affect test outcomes.

10. **Q: What does `'use strict';` do? A:** `'use strict';` is a directive

that enables “strict mode.” Strict mode changes certain JavaScript “bad parts” into explicit errors, makes JavaScript more secure, and optimizes performance. For example, it prevents accidental global variable creation, makes `eval()` safer, and throws errors for silent failures.

11. **Q: Explain the `typeof` operator.** **A:** The `typeof` operator returns a string indicating the type of the unevaluated operand. `javascript`

```
typeof 42;           // "number"    typeof "hello";    //
"string"    typeof true;           // "boolean"    typeof
undefined;   // "undefined"    typeof { a: 1 };    //
"object"    typeof [1, 2];         // "object" (arrays are
objects)    typeof function(){}; // "function" (functions
are objects too)    typeof null;           // "object" (this
is a historical bug)    typeof Symbol();    // "symbol"
```
12. **Q: What is NaN? How can you check if a value is NaN?** **A:** NaN stands for “Not a Number.” It’s a special numeric value that results from an operation that cannot produce a meaningful numerical result (e.g., `0/0`, `Math.sqrt(-1)`). You can check for NaN using `Number.isNaN()` (preferred) or `isNaN()`. NaN is the only value in JavaScript not equal to itself (`NaN === NaN` is `false`). `javascript`

```
console.log(Number.isNaN(0/0));    // true
console.log(isNaN("hello" * 5)); // true
```
13. **Q: What are template literals (template strings)?** **A:** Template literals are string literals allowing embedded expressions, multi-line strings, and string interpolation. They are enclosed by backticks (```). `javascript`

```
const name = "World";    const greeting = `Hello, ${name}!
This is a multi-line string.`;    console.log(greeting);
```
14. **Q: What is an IIFE (Immediately Invoked Function Expression)? Why use it?** **A:** An IIFE is a JavaScript function that runs as soon as it is defined. `javascript`

```
(function() {
  console.log("IIFE executed!");    var localScopedVar =
  "I am local";    })(); Uses:
```

 - **Avoids polluting the global namespace:** Variables declared inside an IIFE are not visible outside, creating a private scope.
 - **Module pattern:** Can be used to create modules with private and public members.
15. **Q: What are JavaScript truthy and falsy values?** **A:**
 - **Falsy values:** Values that evaluate to `false` in a boolean context.
 - `false`
 - `0` (zero)
 - `-0` (minus zero)
 - `0n` (BigInt zero)
 - `""` (empty string)

- null
 - undefined
 - NaN
 - **Truthy values:** All other values are truthy, including empty objects {}, empty arrays [], and non-empty strings.
16. **Q: What are the different types of loops in JavaScript? A:**
- **for:** Loops through a block of code a number of times.
 - **for...in:** Loops through the properties of an object (enumerable, non-Symbol properties).
 - **for...of:** Loops over iterable objects (like Array, Map, Set, String, etc.), giving you the values.
 - **while:** Loops through a block of code while a specified condition is true.
 - **do...while:** Loops through a block of code once, and then repeats the loop while a specified condition is true.
 - Array iteration methods like `forEach`, `map`, `filter`, etc.
17. **Q: What's the difference between `for...in` and `for...of`? A:**
- **for...in** iterates over the *enumerable property names (keys)* of an object. It can also iterate over inherited properties.
 - **for...of** (ES6) iterates over the *values* of an iterable object (e.g., Array, String, Map, Set). It does not work directly on plain objects because they are not inherently iterable.
18. **Q: What is the use of the `break` and `continue` statements? A:**
- **break:** Terminates the current loop, `switch`, or label statement and transfers program control to the statement following the terminated statement.
 - **continue:** Terminates execution of the statements in the current iteration of the current or labeled loop, and continues execution of the loop with the next iteration.
19. **Q: What is short-circuiting in JavaScript? A:** Short-circuiting refers to how logical operators (`&&`, `||`) evaluate expressions.
- **&& (AND):** If the first operand is falsy, it returns that operand without evaluating the second. If the first is truthy, it evaluates and returns the second operand.
 - **|| (OR):** If the first operand is truthy, it returns that operand without evaluating the second. If the first is falsy, it evaluates and returns the second operand.

```
let x = true && "Hello"; // x is "Hello"
let y = false && "Hello"; // y is false
let z = true || "World"; // z is true
let w = false || "World"; // w is "World"
```

20. **Q: What is the conditional (ternary) operator?** **A:** The ternary operator is a shorthand for an `if...else` statement. It takes three operands: a condition followed by a question mark (`?`), then an expression to execute if the condition is truthy, followed by a colon (`:`), and finally the expression to execute if the condition is falsy. `javascript`
- ```
const age = 20; const canVote = (age >= 18) ? "Yes" :
"No"; console.log(canVote); // "Yes"
```
- 

## II. Functions (21-35)

21. **Q: What are the ways to declare a function in JavaScript?** **A:**
- **Function Declaration (Statement):** `javascript`

```
function
greet(name) { return `Hello, ${name}!`; }
}
```
  - **Function Expression (Anonymous or Named):** `javascript`

```
const greet = function(name) { // Anonymous return
`Hello, ${name}!`; }; const greetNamed = function
sayHello(name) { // Named return `Hello, ${name}!`; };
};
```
  - **Arrow Function (ES6):** `javascript`

```
const greet =
(name) => `Hello, ${name}!`;
```
  - **Function Constructor (rarely used):** `javascript`

```
const
greet = new Function('name', 'return `Hello, ${name}!`;');
```
22. **Q: What is the difference between a function declaration and a function expression?** **A:**
- **Hoisting:** Function declarations are hoisted completely (name and body). Function expressions are only partially hoisted if declared with `var` (variable name hoisted, initialized to `undefined`), or not initialized if declared with `let/const` (TDZ).
  - **Usage:** Function declarations can be called before they are defined in the code. Function expressions must be defined before they are called (unless `var` is used, then it's `undefined`).
  - **Conditional Definition:** Function expressions can be conditionally defined (e.g., inside an `if` block), whereas function declarations in blocks can behave inconsistently across environments (strict mode helps).
23. **Q: What are arrow functions? How do they differ from regular functions?** **A:** Arrow functions (ES6) provide a concise syntax for writing functions. Differences:
- **Syntax:** Shorter syntax.
  - **this binding:** Arrow functions do not have their own `this` context. They lexically inherit `this` from the surrounding (parent) scope.
  - **arguments object:** Arrow functions do not have their own `arguments` object. You can use rest parameters (`...args`) instead.

- **new keyword:** Cannot be used as constructors (cannot be called with `new`).
  - **prototype property:** Do not have a `prototype` property.
  - **No `super` or `new.target`:** They don't have their own bindings for these.
24. **Q: What is the `this` keyword in JavaScript? How does its value get determined? A:** The `this` keyword refers to the context in which a function is executed. Its value is determined by how the function is called (invocation context):
- **Global Context:** In the global execution context (outside any function), `this` refers to the global object (`window` in browsers, `global` in Node.js). In strict mode, `this` is `undefined` in the global context when not assigned.
  - **Function Context (Default Binding):** When a regular function is called directly (not as a method of an object), `this` usually refers to the global object. In strict mode, `this` is `undefined`.
  - **Method Invocation (Implicit Binding):** When a function is called as a method of an object, `this` refers to the object the method is called on.
  - **Constructor Invocation (New Binding):** When a function is called with the `new` keyword, `this` refers to the newly created instance.
  - **Explicit Binding (`call`, `apply`, `bind`):** These methods allow you to explicitly set the value of `this` for a function call.
  - **Arrow Functions:** `this` is lexically bound; it takes the `this` value of its surrounding lexical scope.
25. **Q: Explain `call()`, `apply()`, and `bind()`. A:** These are methods of the `Function.prototype` used to control the `this` context of a function and pass arguments.
- **`call(thisArg, arg1, arg2, ...)`:** Invokes the function immediately with a specified `this` value and arguments provided individually.
  - **`apply(thisArg, [argsArray])`:** Invokes the function immediately with a specified `this` value and arguments provided as an array (or an array-like object).
  - **`bind(thisArg, arg1, arg2, ...)`:** Returns a *new function* where `this` is permanently bound to `thisArg`, and optionally, some initial arguments are pre-set (currying). The new function can be called later.
26. **Q: What is a closure? Give an example and a use case. A:** A closure is a function that has access to its own scope, the scope of the outer function, and the global scope. Essentially, a closure “remembers” the environment (variables) in which it was created, even after the outer

function has finished executing.

```
function outerFunction(outerVariable) {
 return function innerFunction(innerVariable) {
 console.log("Outer variable: " + outerVariable);
 console.log("Inner variable: " + innerVariable);
 };
}

const newFunction = outerFunction("outside");
newFunction("inside");
// Output:
// Outer variable: outside
// Inner variable: inside
```

Use Cases:

- **Data Encapsulation / Private Variables:** Creating private variables that can only be accessed through specific methods.
- **Callbacks and Event Handlers:** Maintaining state in asynchronous operations.
- **Currying and Partial Application.**
- **Module Pattern.**

27. **Q: What are higher-order functions? A:** Higher-order functions are functions that either:

1. Take one or more functions as arguments.
2. Return a function as their result. Examples include `map`, `filter`, `reduce`, `setTimeout`, and functions that return other functions (like in closures or currying).

28. **Q: What are pure functions? Why are they useful? A:** A pure function is a function that:

1. Given the same input, will always return the same output.
2. Produces no side effects (e.g., doesn't modify external state, log to console, mutate its input arguments if they are objects/arrays). **Usefulness:**

- **Predictability:** Easier to reason about and understand.
- **Testability:** Simple to test as output only depends on input.
- **Memoization/Caching:** Results can be cached.
- **Parallelism:** Can be run in parallel without interference.

29. **Q: What is function currying? A:** Currying is a technique of transforming a function that takes multiple arguments into a sequence of functions, each taking a single argument. Each function returns another function until all arguments are supplied, at which point the final result is returned.

```
function add(a) {
 return function(b) {
 return function(c) {
 return a + b + c;
 };
 };
}
console.log(add(1)(2)(3)); // 6

// Or with arrow functions:
const addArrow = a => b => c => a + b + c;
console.log(addArrow(1)(2)(3)); // 6
```

30. **Q: What are default parameters in functions?** **A:** Default parameters (ES6) allow you to initialize named parameters with default values if no value or undefined is passed. `javascript` `function`
- ```
greet(name = "Guest", greeting = "Hello") { console.log(`${greeting},
${name}!`); } greet(); // "Hello,
Guest!" greet("Alice"); // "Hello, Alice!"
greet("Bob", "Hi"); // "Hi, Bob!"
```
31. **Q: What are rest parameters?** **A:** Rest parameters (ES6, syntax: `...paramName`) allow a function to accept an indefinite number of arguments as an array. It collects all remaining arguments passed to the function into a single array. It must be the last parameter in the function definition. `javascript` `function`
- ```
sumAll(...numbers) { //
 numbers will be an array return numbers.reduce((sum,
 num) => sum + num, 0); } console.log(sumAll(1, 2,
 3)); // 6 console.log(sumAll(10, 20, 30, 40)); // 100
```
32. **Q: What is the arguments object? How does it differ from rest parameters?** **A:** The `arguments` object is an array-like object (not a true array) accessible inside regular functions (not arrow functions) that contains the values of the arguments passed to that function. Differences from rest parameters:
- **Type:** `arguments` is array-like; rest parameters create a true array.
  - **Availability:** `arguments` is not available in arrow functions.
  - **Specificity:** Rest parameters capture only the “rest” of the arguments not explicitly named, while `arguments` captures all of them.
  - **Clarity:** Rest parameters are generally more explicit and easier to use.
33. **Q: What is recursion? Give an example.** **A:** Recursion is a programming technique where a function calls itself directly or indirectly to solve a problem. A recursive function must have a base case to stop the recursion and prevent an infinite loop (stack overflow). `javascript` `// Factorial example` `function`



```

factorial(n) { if (n === 0 || n === 1) { // Base
case return 1; } else { // Recursive
step return n * factorial(n - 1); }
} console.log(factorial(5)); // 120

```

34. **Q: What is a callback function?** **A:** A callback function is a function passed into another function as an argument, which is then invoked inside the outer function to complete some kind of routine or action. Callbacks are fundamental to asynchronous programming in JavaScript.
- ```

javascript      function processData(data,
callback) {      // ... processing data ...      const
result = data.toUpperCase();      callback(result);
}      processData("hello", function(processedResult) {
console.log("Processed: " + processedResult); // Processed:
HELLO      });

```
35. **Q: What is “callback hell” and how can it be avoided?** **A:** “Callback hell” (also known as the Pyramid of Doom) refers to deeply nested callback functions, which can make code hard to read, debug, and maintain. Avoidance strategies:
- **Named Functions:** Break callbacks into smaller, named functions.
 - **Modularity:** Create modules to handle specific parts of the asynchronous logic.
 - **Promises (ES6):** Provide a cleaner way to handle asynchronous operations and chaining.
 - **Async/Await (ES7+):** Syntactic sugar on top of Promises, making asynchronous code look more synchronous.

III. Objects & Arrays (36-55)

36. **Q: How do you create an object in JavaScript?** **A:**

- **Object Literal:** `javascript const person = { name: "Alice", age: 30 };`
- **new Object() Constructor:** `javascript const person = new Object(); person.name = "Bob"; person.age = 25;`
- **Constructor Function:** `javascript function Person(name, age) { this.name = name; this.age = age; } const person = new Person("Charlie", 35);`
- **Object.create():** Creates a new object with a specified prototype object. `javascript const proto = { greet: function() { console.log("Hello!"); } }; const person = Object.create(proto); person.name = "David";`
- **ES6 Classes:** `javascript class Person { constructor(name, age) { this.name = name; this.age`

```

    = age;          }      }      const person = new Person("Eve",
28);

```

37. **Q: How do you access properties of an object? A:**

- **Dot Notation:** `object.propertyName` javascript `const person = { name: "Alice" }; console.log(person.name);`
// Alice
- **Bracket Notation:** `object['propertyName']` (useful when property name is dynamic or has special characters) javascript `const person = { "full-name": "Bob Smith" }; console.log(person['full-name']);`
// Bob Smith `const prop = "full-name"; console.log(person[prop]);`
// Bob Smith

38. **Q: How do you iterate over an object's properties? A:**

- **for...in loop:** Iterates over enumerable property names (keys), including inherited ones. Use `hasOwnProperty` to check for own properties. javascript `const obj = { a: 1, b: 2 }; for (const key in obj) { if (obj.hasOwnProperty(key)) { console.log(key, obj[key]); } }`
- **Object.keys(obj):** Returns an array of an object's own enumerable property names.
- **Object.values(obj):** Returns an array of an object's own enumerable property values.
- **Object.entries(obj):** Returns an array of an object's own enumerable [key, value] pairs. javascript `Object.keys(obj).forEach(key => console.log(key, obj[key]));` `Object.values(obj).forEach(value => console.log(value));` `Object.entries(obj).forEach(([key, value]) => console.log(key, value));`

39. **Q: What is the `hasOwnProperty` method? A:** `obj.hasOwnProperty(prop)` returns a boolean indicating whether the object has the specified property as its *own* property (as opposed to inheriting it).

40. **Q: How do you create an array in JavaScript? A:**

- **Array Literal:** javascript `const fruits = ["Apple", "Banana", "Cherry"];`
- **new Array() Constructor:** javascript `const numbers = new Array(1, 2, 3);` `const emptyArr = new Array(5);`
// Creates an array with 5 empty slots
- **Array.from():** Creates a new array instance from an array-like or iterable object.
- **Array.of():** Creates a new array instance with a variable number of arguments, regardless of number or type.

41. **Q: What are some common array methods? (Mention at least 5) A:**

- **push():** Adds one or more elements to the end of an array and returns the new length.
- **pop():** Removes the last element from an array and returns that element.
- **shift():** Removes the first element from an array and returns that element.
- **unshift():** Adds one or more elements to the beginning of an array and returns the new length.
- **slice(start, end):** Returns a shallow copy of a portion of an array into a new array object. Original array is not modified.
- **splice(start, deleteCount, ...itemsToAdd):** Changes the contents of an array by removing or replacing existing elements and/or adding new elements in place. Modifies the original array.
- **concat():** Merges two or more arrays. Returns a new array.
- **join(separator):** Joins all elements of an array into a string.
- **indexOf(element):** Returns the first index at which a given element can be found, or -1 if not present.
- **includes(element):** Determines whether an array includes a certain value, returning `true` or `false`.

42. **Q: Explain `forEach`, `map`, `filter`, and `reduce` array methods. A:**

These are higher-order functions for array iteration and transformation:

- **forEach(callbackFn):** Executes a provided function once for each array element. Does not return a new array; its return value is `undefined`. Primarily used for side effects. `javascript` `[1, 2, 3].forEach(num => console.log(num * 2)); // Logs 2, 4, 6`
- **map(callbackFn):** Creates a *new array* populated with the results of calling a provided function on every element in the calling array. `javascript` `const numbers = [1, 2, 3]; const doubled = numbers.map(num => num * 2); // doubled is [2, 4, 6]`
- **filter(callbackFn):** Creates a *new array* with all elements that pass the test implemented by the provided function. `javascript` `const numbers = [1, 2, 3, 4, 5]; const evens = numbers.filter(num => num % 2 === 0); // evens is [2, 4]`
- **reduce(callbackFn, initialValue):** Executes a “reducer” function on each element of the array, resulting in a single output value. The reducer function takes an accumulator and the current value as arguments. `javascript` `const numbers = [1, 2, 3, 4]; const sum = numbers.reduce((accumulator, currentValue) => accumulator + currentValue, 0); // sum is 10`

43. **Q: What is array destructuring? Give an example. A:** Array destructuring (ES6) is a syntax that allows unpacking values from arrays, or properties from objects, into distinct variables.

```

const numbers = [10, 20, 30, 40, 50];

// Basic destructuring
const [first, second] = numbers;
console.log(first); // 10
console.log(second); // 20

// Skipping elements
const [, , third] = numbers;
console.log(third); // 30

// Using rest operator
const [one, two, ...rest] = numbers;
console.log(one); // 10
console.log(two); // 20
console.log(rest); // [30, 40, 50]

```

44. **Q: What is object destructuring? Give an example. A:** Object destructuring (ES6) allows unpacking properties from objects into distinct variables.

```

const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30,
  address: {
    city: "New York",
    country: "USA"
  }
};

// Basic destructuring
const { firstName, age } = person;
console.log(firstName); // John
console.log(age); // 30

// Assigning to new variable names
const { lastName: surname } = person;
console.log(surname); // Doe

// Nested destructuring
const { address: { city } } = person;
console.log(city); // New York

// Default values
const { middleName = "N/A" } = person;
console.log(middleName); // N/A

```

45. **Q: What is the spread operator (...)? How is it used with arrays and objects? A:** The spread operator (ES6) allows an iterable (like an array or string) to be expanded in places where zero or more arguments (for function calls) or elements (for array literals) are expected. For objects (ES2018+), it copies an object's own enumerable properties into a new object.

- **With Arrays:**

```
const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];
const combined = [...arr1, ...arr2, 7]; // [1, 2, 3, 4, 5, 6, 7] - Concatenation
const copy = [...arr1];                // [1, 2, 3] - Shallow copy

function sum(x, y, z) { return x + y + z; }
sum(...arr1); // Equivalent to sum(1, 2, 3)
```

- **With Objects (ES2018+):** javascript const obj1 = {
a: 1, b: 2 }; const obj2 = { b: 3, c: 4 }; const
merged = { ...obj1, ...obj2 }; // { a: 1, b: 3, c: 4
} (obj2's b overwrites obj1's) const cloned = {
...obj1 }; // { a: 1, b: 2 } - Shallow clone

46. **Q: How can you check if a variable is an array? A:** The most reliable way is `Array.isArray()`: javascript const arr = [1, 2, 3];
const obj = { a: 1 }; console.log(Array.isArray(arr));
// true console.log(Array.isArray(obj)); // false Using
`instanceof Array` can also work but might fail across different <iframe> or realm contexts. `typeof []` returns "object", which is not specific enough.

47. **Q: What is the difference between shallow copy and deep copy of an object/array? How can you achieve them? A:**

- **Shallow Copy:** Creates a new object/array, but if it contains nested objects/arrays, only references to those nested structures are copied, not the structures themselves. Modifying a nested object in the copy will also affect the original.
 - Achieved using: Spread operator (...), `Object.assign()`, `Array.prototype.slice()`, `Array.from()`.
- **Deep Copy:** Creates a new object/array and recursively copies all nested objects/arrays, so the copy is completely independent of the original.
 - Achieved using:
 - * `JSON.parse(JSON.stringify(obj))` (Simple but has limitations: loses functions, `undefined`, `Date` objects become strings, etc.)
 - * Libraries like `Lodash` (`_.cloneDeep()`).
 - * Custom recursive function.

* `structuredClone()` (modern browser API for deep cloning)

48. **Q: What are `Object.freeze()`, `Object.seal()`, and `Object.preventExtensions()`?**

A:

- **`Object.preventExtensions(obj)`:** Prevents new properties from being added to an object. Existing properties can still be modified or deleted.
- **`Object.seal(obj)`:** Prevents new properties from being added and marks all existing properties as non-configurable (meaning they cannot be deleted, and their descriptors cannot be changed, but their values can still be changed if they are writable).
- **`Object.freeze(obj)`:** Prevents new properties from being added, makes all existing properties non-configurable, and makes all data properties non-writable. The object becomes effectively immutable (at a shallow level; nested objects are not automatically frozen).

49. **Q: What is JSON? What are `JSON.stringify()` and `JSON.parse()`?**

A:

- **JSON (JavaScript Object Notation):** A lightweight data-interchange format. It's easy for humans to read and write and easy for machines to parse and generate. It's based on a subset of JavaScript syntax but is language-independent.
- **`JSON.stringify(value)`:** Converts a JavaScript value (object, array, primitive) into a JSON string. Functions and `undefined` values are typically omitted or converted to `null` (in arrays).
- **`JSON.parse(text)`:** Parses a JSON string, constructing the JavaScript value or object described by the string.

50. **Q: What are JavaScript Map and Set objects (ES6)?** **A:**

- **Set:** A collection of unique values of any type. Values in a `Set` can only occur once.

```
javascript    const mySet = new Set([1, 2, 2, 3, "hello", "hello"]);    console.log(mySet); // Set(4) { 1, 2, 3, 'hello' }    mySet.add(4);    mySet.has(2); // true
```
- **Map:** A collection of key-value pairs where keys can be of any data type (unlike object literals where keys are strings or Symbols). Remembers the original insertion order of the keys.

```
javascript    const myMap = new Map();    const keyObj = {};    myMap.set("name", "Alice");    myMap.set(keyObj, "An object key");    console.log(myMap.get("name")); // Alice    console.log(myMap.get(keyObj)); // An object key
```

51. **Q: How to find an element in an array?** **A:**

- **`indexOf(element, fromIndex)`:** Returns the first index of an element, or `-1`.

- **lastIndexOf(element, fromIndex):** Returns the last index of an element, or -1.
- **includes(element, fromIndex):** Returns **true** if element is found, **false** otherwise. (ES7)
- **find(callbackFn):** Returns the *value* of the first element that satisfies the testing function, or **undefined**.
- **findIndex(callbackFn):** Returns the *index* of the first element that satisfies the testing function, or -1.

```
const arr = [10, 20, 30, 20];
console.log(arr.indexOf(20)); // 1
console.log(arr.includes(30)); // true
const found = arr.find(el => el > 25); // 30
```

52. **Q: How to remove an element from an array? A:**

- **pop():** Removes the last element.
- **shift():** Removes the first element.
- **splice(startIndex, deleteCount):** Removes elements from a specific index. Modifies the original array. **javascript** `const arr = [1, 2, 3, 4, 5]; arr.splice(2, 1);` // Removes 1 element starting at index 2. **arr** is now [1, 2, 4, 5]
- **filter() (to create a new array without the element):** **javascript** `const arr = [1, 2, 3, 2, 4]; const newArr = arr.filter(el => el !== 2);` // **newArr** is [1, 3, 4]

53. **Q: How to check if an object is empty? A:** **javascript**

```
function isEmptyObject(obj) {
  if (obj === null ||
    typeof obj !== 'object') return true; // Or handle as
  an error
  return Object.keys(obj).length === 0 &&
    obj.constructor === Object;
}
console.log(isEmptyObject({})); // true
console.log(isEmptyObject({a:1})); // false
console.log(isEmptyObject(new Date())); // false (constructor
check) A simpler check if you only care about own enumerable properties:
Object.keys(obj).length === 0
```

54. **Q: What does Object.keys(), Object.values(), Object.entries() do? A:**

- **Object.keys(obj):** Returns an array of a given object's own enumerable property *names* (keys).
- **Object.values(obj):** Returns an array of a given object's own enumerable property *values*.
- **Object.entries(obj):** Returns an array of a given object's own enumerable string-keyed property *[key, value]* pairs.

55. **Q: How can you create an immutable object or array in JavaScript (conceptually)? A:** True immutability is hard to enforce

deeply without libraries.

- **Object.freeze():** Provides shallow immutability for objects. For arrays, it prevents adding/removing elements, but elements themselves (if objects/arrays) can still be mutated.
- **Discipline/Convention:** Always create new objects/arrays instead of modifying existing ones when making changes (e.g., using spread operator, `slice`, `map`, `filter`).
- **Libraries:** Use libraries like Immer or Immutable.js which provide efficient immutable data structures.

```
// Conceptual example with spread for arrays
const originalArray = [1, 2, { val: 3 }];
const newArray = [...originalArray, 4]; // Adding an element
const modifiedArray = originalArray.map(item =>
  item.val === 3 ? { ...item, val: 30 } : item
); // "Modifying" a nested object immutably
```

IV. Asynchronous JavaScript (56-70)

56. **Q: What is synchronous vs. asynchronous programming? A:**

- **Synchronous:** Code execution happens in sequence. Each operation must complete before the next one starts. If an operation takes a long time, it blocks further execution.
- **Asynchronous:** Code execution does not necessarily happen in sequence. Operations (especially I/O-bound like network requests, file system operations) can be initiated, and the program can continue to execute other tasks while waiting for these operations to complete. When an async operation finishes, a callback, Promise, or other mechanism handles the result.

57. **Q: Explain the JavaScript Event Loop. A:** The JavaScript Event Loop is a mechanism that allows Node.js or the browser to perform non-blocking I/O operations despite JavaScript being single-threaded. It consists of:

- **Call Stack:** Where JavaScript code is executed. Functions are pushed onto the stack when called and popped off when they return.
- **Web APIs / C++ APIs (Node.js):** Handles asynchronous operations (e.g., `setTimeout`, DOM events, `fetch`). When an async task completes, its callback is placed in a queue.
- **Callback Queue (Task Queue / Message Queue):** Holds callback functions ready to be executed once the Call Stack is empty.
- **Microtask Queue (Job Queue):** Holds callbacks from Promises (`.then`, `.catch`, `.finally`) and `MutationObserver`. Microtasks have higher priority and are executed after the current script and before the browser renders or any new tasks from the Callback Queue are

processed. The Event Loop continuously checks if the Call Stack is empty. If it is, it takes the first task from the Microtask Queue (if any) and pushes it onto the Call Stack. If the Microtask Queue is empty, it takes a task from the Callback Queue.

58. **Q: What are Promises? What states can a Promise be in? A:** A Promise is an object representing the eventual completion (or failure) of an asynchronous operation and its resulting value. It's a placeholder for a future value. States:

1. **pending:** Initial state, neither fulfilled nor rejected.
2. **fulfilled (or resolved):** The operation completed successfully, and the Promise has a resulting value.
3. **rejected:** The operation failed, and the Promise has a reason for the failure. A promise is *settled* if it's either fulfilled or rejected (i.e., not pending).

59. **Q: How do you use Promises? (e.g., then, catch, finally) A:**

- **promise.then(onFulfilled, onRejected):** Attaches callbacks to handle the fulfilled or rejected state of the Promise. **onFulfilled** is called with the resolved value, and **onRejected** (optional) is called with the rejection reason. Returns a new Promise, allowing chaining.
- **promise.catch(onRejected):** A shorthand for **promise.then(null, onRejected)**. Attaches a callback to handle only the rejected state. Also returns a new Promise.
- **promise.finally(onFinally):** Attaches a callback that is executed when the Promise is settled (either fulfilled or rejected). It receives no arguments. Useful for cleanup. Returns a new Promise.

```
function fetchData() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      const success = Math.random() > 0.5;
      if (success) {
        resolve("Data fetched successfully!");
      } else {
        reject(new Error("Failed to fetch data."));
      }
    }, 1000);
  });
}

fetchData()
  .then(data => {
    console.log(data); // "Data fetched successfully!"
    return data.toUpperCase(); // Value passed to next .then
  })
```

```

.then(processedData => {
  console.log("Processed:", processedData);
})
.catch(error => {
  console.error(error.message); // "Failed to fetch data."
})
.finally(() => {
  console.log("Fetch operation finished.");
});

```

60. **Q: What is `async/await`? How does it relate to Promises?** **A:** `async/await` (ES2017) is syntactic sugar built on top of Promises, making asynchronous code look and behave a bit more like synchronous code, which can improve readability.

- **async keyword:** Used to declare an asynchronous function. An `async` function implicitly returns a Promise. If the function returns a value, the Promise will be resolved with that value. If it throws an error, the Promise will be rejected.
- **await keyword:** Can only be used inside an `async` function. It pauses the execution of the `async` function until the Promise it's waiting for is settled (resolved or rejected). If the Promise resolves, `await` returns the resolved value. If it rejects, `await` throws the rejection reason (which can be caught with `try...catch`).

```

async function fetchDataAsync() {
  try {
    console.log("Fetching data...");
    const data = await fetchData(); // fetchData is the Promise-returning function
    console.log(data);
    const processedData = data.toUpperCase();
    console.log("Processed:", processedData);
    return processedData;
  } catch (error) {
    console.error("Error in async function:", error.message);
    throw error; // Re-throw if needed
  } finally {
    console.log("Async fetch operation finished.");
  }
}

fetchDataAsync().then(result => console.log("Final result:", result));

```

61. **Q: How do you handle errors in `async/await`?** **A:** Errors in `async/await` are typically handled using standard `try...catch...finally` blocks, just like synchronous code. When an awaited Promise rejects, it throws an error that can be caught by a `catch` block.

```

async function riskyOperation() {

```

```

    if (Math.random() < 0.5) {
      throw new Error("Something went wrong!");
    }
    return "Success!";
  }
}

async function performTask() {
  try {
    const result = await riskyOperation();
    console.log(result);
  } catch (error) {
    console.error("Caught an error:", error.message);
  }
}

performTask();

```

62. **Q: What is `Promise.all()`?** **A:** `Promise.all(iterable)` takes an iterable (e.g., an array) of Promises and returns a single new Promise. This new Promise fulfills when *all* of the input Promises have fulfilled, with an array of their fulfillment values (in the same order as the input Promises). It rejects immediately if *any* of the input Promises reject, with the reason of the first Promise that rejected.
63. **Q: What is `Promise.race()`?** **A:** `Promise.race(iterable)` takes an iterable of Promises and returns a single new Promise. This new Promise settles (fulfills or rejects) as soon as *one* of the input Promises settles, with the value or reason from that first-settled Promise.
64. **Q: What is `Promise.allSettled()`?** **A:** `Promise.allSettled(iterable)` (ES2020) takes an iterable of Promises and returns a single new Promise. This new Promise fulfills when *all* of the input Promises have settled (either fulfilled or rejected). The fulfillment value is an array of objects, each describing the outcome of one Promise in the input iterable. Each result object has a **status** ("fulfilled" or "rejected") and either a **value** (if fulfilled) or a **reason** (if rejected). Unlike `Promise.all()`, it doesn't short-circuit on rejection.
65. **Q: What is `Promise.any()`?** **A:** `Promise.any(iterable)` (ES2021) takes an iterable of Promises and returns a single new Promise. This new Promise fulfills as soon as *one* of the input Promises fulfills, with the value of that first fulfilled Promise. If *all* of the input Promises reject, then the returned Promise is rejected with an **AggregateError**, a new type of error object that groups together individual errors.
66. **Q: Can you explain `setTimeout(callback, delay)` and `setInterval(callback, delay)`?** **A:**
 - **`setTimeout(callback, delay)`:** Executes a `callback` function once after a specified `delay` (in milliseconds). It returns a timeout

ID which can be used with `clearTimeout()` to cancel the timeout.

- **setInterval(callback, delay):** Repeatedly executes a `callback` function with a fixed `delay` (in milliseconds) between each call. It returns an interval ID which can be used with `clearInterval()` to stop the repetitions.

67. **Q: What's the difference between the Microtask Queue and the Macrotask (Callback) Queue? A:**

- **Macrotask Queue (Task Queue / Callback Queue):** Holds tasks like `setTimeout`, `setInterval` callbacks, I/O operations, UI rendering (in browsers).
- **Microtask Queue (Job Queue):** Holds tasks like Promise callbacks (`.then`, `.catch`, `.finally`), `queueMicrotask()`, `MutationObserver` callbacks. **Execution Order:** The Event Loop prioritizes the Microtask Queue. After each macrotask from the Callback Queue finishes (and the call stack becomes empty), the Event Loop processes *all* tasks currently in the Microtask Queue before moving on to the next macrotask or rendering. This means microtasks can starve macrotasks if they keep adding more microtasks.

68. **Q: What is fetch() API? How does it differ from XMLHttpRequest?**

A: The `fetch()` API is a modern JavaScript interface for making network requests (e.g., HTTP requests to servers). It's Promise-based. Differences from `XMLHttpRequest` (XHR):

- **Promises:** `fetch()` returns a Promise, making it easier to work with `async/await` and chain operations. XHR is event-based.
- **Simpler API:** `fetch()` has a cleaner and more straightforward API for common use cases.
- **CORS:** `fetch()` handles CORS more gracefully by default (e.g., opaque responses for cross-origin no-CORS requests).
- **Streaming:** `fetch()` supports Request and Response streaming.
- **Error Handling:** `fetch()` only rejects its Promise on network errors (e.g., DNS failure, no connection). HTTP error statuses (like 404 or 500) are *not* considered network errors, so the Promise resolves. You need to check `response.ok` or `response.status`. XHR handles HTTP errors through its `onerror` or by checking status in `onreadystatechange`.

69. **Q: How do you make a POST request using fetch()? A:**

```
async function postData(url = '', data = {}) {
  const response = await fetch(url, {
    method: 'POST', // *GET, POST, PUT, DELETE, etc.
    headers: {
      'Content-Type': 'application/json'
      // 'Content-Type': 'application/x-www-form-urlencoded',
    }
  })
}
```

```

    },
    body: JSON.stringify(data) // body data type must match "Content-Type" header
  });
  if (!response.ok) {
    throw new Error(`HTTP error! status: ${response.status}`);
  }
  return response.json(); // parses JSON response into native JavaScript objects
}

postData('https://api.example.com/submit', { item: 'test' })
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.error('Error:', error);
  });

```

70. **Q: What are Web Workers? Why use them? A:** Web Workers allow you to run JavaScript code in a background thread, separate from the main browser thread (which handles UI updates and user interactions). **Why use them?**

- **Prevent UI Freezing:** Offload long-running or CPU-intensive tasks to a worker thread, preventing the main thread from becoming unresponsive and the UI from freezing.
- **Improved Performance:** Can lead to better perceived performance and smoother user experience for complex applications. Workers communicate with the main thread using `postMessage()` and the `onmessage` event handler. They do not have direct access to the DOM.

V. OOP & Prototypes (71-80)

71. **Q: What is prototypal inheritance in JavaScript? A:** Prototypal inheritance is a feature in JavaScript where objects can inherit properties and methods from other objects (their “prototype”). Each object has a private property (often accessible via `__proto__` or `Object.getPrototypeOf()`) that links to its prototype. If a property is accessed on an object and not found, JavaScript looks up the prototype chain until the property is found or the chain ends (at `null`).
72. **Q: What is the prototype property on functions? A:** Every JavaScript function (except arrow functions) has a `prototype` property. This property is an object that becomes the prototype for all instances created when that function is used as a constructor (with the `new` keyword). Methods and properties added to a constructor function’s `prototype` object are shared among all instances created by that constructor.

73. **Q: What is the difference between `__proto__` (or `[[Prototype]]`) and `prototype`? A:**

- **`prototype`:** Is a property of a *constructor function*. It's the object that will become the prototype for objects created by that constructor.
- **`__proto__` (or `[[Prototype]]` internal slot, accessible via `Object.getPrototypeOf()`):** Is an internal property of an *object instance*. It's a reference to the object's prototype (i.e., the constructor function's `prototype` object from which it was instantiated, or another object if set via `Object.create()`).

74. **Q: How does the `new` keyword work? A:** When `new` is used with a constructor function (e.g., `new Person(...)`):

1. A new, empty plain JavaScript object is created.
2. This new object's `[[Prototype]]` (or `__proto__`) is linked to the constructor function's `prototype` object.
3. The constructor function is called with `this` bound to the newly created object.
4. If the constructor function does not explicitly return an object, the newly created object (`this`) is returned. Otherwise, the explicitly returned object is used.

75. **Q: What are ES6 Classes? How do they relate to prototypal inheritance? A:** ES6 Classes provide a cleaner, more familiar syntax (for those coming from class-based languages) for creating objects and implementing inheritance. They are primarily syntactic sugar over JavaScript's existing prototypal inheritance mechanism.

```
class Animal {
  constructor(name) {
    this.name = name;
  }
  speak() {
    console.log(`${this.name} makes a noise.`);
  }
}

class Dog extends Animal { // 'extends' sets up the prototype chain
  constructor(name, breed) {
    super(name); // Calls the parent class's constructor
    this.breed = breed;
  }
  speak() { // Overrides parent method
    console.log(`${this.name} barks.`);
  }
}
```

```
const myDog = new Dog("Rex", "German Shepherd");
myDog.speak(); // Rex barks.
console.log(myDog instanceof Dog); // true
console.log(myDog instanceof Animal); // true
```

Under the hood, `Dog.prototype` will inherit from `Animal.prototype`.

76. Q: Explain `super()` in ES6 classes. A:

- In a subclass constructor, `super()` must be called *before* `this` can be used. It calls the constructor of the parent class and binds `this` to the new instance correctly.
- In a subclass method, `super.methodName()` can be used to call the corresponding method on the parent class.

77. Q: What are static methods and properties in ES6 classes? A:

Static methods and properties belong to the class itself, not to instances of the class. They are called directly on the class, not on an instance. They are often utility functions or properties related to the class but not specific to any instance.

```
class MathHelper {
  static PI = 3.14159; // Static property (ES2022 proposal, widely supported)

  static add(x, y) { // Static method
    return x + y;
  }
}

console.log(MathHelper.PI); // 3.14159
console.log(MathHelper.add(5, 3)); // 8
// const helper = new MathHelper();
// helper.add(1,1) // This would be an error
```

78. Q: Can you achieve private members in ES6 classes? A: Yes, with ES2022 private class fields:

- **Private instance fields:** Declared with a `#` prefix (e.g., `#myPrivateField`). They are only accessible from within the class.
- **Private methods:** Also declared with a `#` prefix (e.g., `#myPrivateMethod()`).

```
class Counter {
  #count = 0; // Private instance field

  constructor(initialCount = 0) {
    this.#count = initialCount;
  }

  #increment() { // Private method
```

```

        this.#count++;
    }

    incrementPublic() {
        this.#increment();
    }

    getCount() {
        return this.#count;
    }
}

const c = new Counter(5);
c.incrementPublic();
console.log(c.getCount()); // 6
// console.log(c.#count); // SyntaxError: Private field '#count' must be declared in an enclosing class
// c.#increment();        // SyntaxError

```

Before this feature, “privacy” was often achieved through conventions (e.g., `_` prefix) or closures (in factory functions/module pattern).

79. **Q: What is method overriding in classes?** **A:** Method overriding occurs when a subclass provides a specific implementation for a method that is already defined in its superclass. The subclass’s method “overrides” the superclass’s method for instances of the subclass. `javascript`
- ```

class Animal {
 speak() { console.log("Animal sound"); }
}
class Dog extends Animal {
 speak() { console.log("Woof!"); } // Overrides Animal's speak
}
const dog = new Dog();
dog.speak(); // Woof!

```
80. **Q: What is the instanceof operator?** **A:** The instanceof operator tests whether the prototype property of a constructor appears anywhere in the prototype chain of an object. `javascript`
- ```

function Car() {}
const myCar = new Car();
console.log(myCar instanceof Car); // true
console.log(myCar instanceof Object); // true (Object is at the top of the prototype chain)
console.log([] instanceof Array); // true
console.log([] instanceof Object); // true

```

VI. DOM & Browser APIs (81-90)

81. **Q: What is the DOM (Document Object Model)?** **A:** The DOM is a programming interface for web documents (HTML, XML). It represents the page structure as a tree of objects (nodes), where each node represents a part of the document (e.g., an element, text, comment). JavaScript can interact with the DOM to dynamically read and modify the content, structure, and style of a web page.

82. **Q: How do you select an HTML element using JavaScript?**
(Mention a few ways) **A:**

- `document.getElementById('elementId')`: Selects an element by its ID.
- `document.getElementsByClassName('className')`: Returns an `HTMLCollection` of elements with the given class name.
- `document.getElementsByTagName('tagName')`: Returns an `HTMLCollection` of elements with the given tag name.
- `document.querySelector('cssSelector')`: Returns the *first* element that matches the specified CSS selector.
- `document.querySelectorAll('cssSelector')`: Returns a static `NodeList` representing a list of elements that match the specified CSS selector.

83. **Q: What's the difference between an `HTMLCollection` and a `NodeList`?** **A:**

- **HTMLCollection:** A *live* collection of elements. If the DOM changes (e.g., an element is added or removed that matches the collection's criteria), the `HTMLCollection` updates automatically. It typically contains only `Element` nodes. Methods like `getElementsByClassName` and `getElementsByTagName` return `HTMLCollections`.
- **NodeList:** Can be *live* or *static*.
 - `document.childNodes` is live.
 - `document.querySelectorAll()` returns a *static* `NodeList`. This means it's a snapshot of the elements at the time the query was made and doesn't update if the DOM changes.
 - `NodeLists` can contain any node type (`Element`, `Text`, `Comment` nodes), not just `Element` nodes. Both are array-like but not true arrays (though they have a `length` property and can be iterated). You can convert them to arrays using `Array.from()` or the spread operator.

84. **Q: How do you add/remove/modify attributes of an HTML element?** **A:**

- **Get Attribute:** `element.getAttribute('attributeName')`
- **Set Attribute:** `element.setAttribute('attributeName', 'value')`
- **Remove Attribute:** `element.removeAttribute('attributeName')`
- **Check for Attribute:** `element.hasAttribute('attributeName')`
- For common attributes like `id`, `class`, `src`, you can also access them as direct properties: `element.id`, `element.className`, `element.src`.

85. **Q: How do you change the content of an HTML element?** **A:**

- **element.innerHTML:** Gets or sets the HTML content (markup) within an element. Can be a security risk if setting content from untrusted sources (XSS).
- **element.textContent:** Gets or sets the text content of an element and its descendants, ignoring HTML tags. Safer for setting plain text.
- **element.innerText:** Similar to `textContent` but is aware of CSS styling (e.g., won't include text from hidden elements) and can be slower. `textContent` is generally preferred for performance and consistency.

86. **Q: How do you add an event listener to an HTML element?**

A: Using `element.addEventListener(eventType, listenerFunction, useCaptureOrOptions)`:

```
const myButton = document.getElementById('myBtn');
function handleClick() {
    console.log('Button clicked!');
}
myButton.addEventListener('click', handleClick);
```

```
// To remove:
// myButton.removeEventListener('click', handleClick);
```

`eventType` is a string like `'click'`, `'mouseover'`, `'keydown'`. `listenerFunction` is the function to call when the event occurs. `useCaptureOrOptions` can be a boolean (for capture phase) or an options object (e.g., `{ capture: true, once: true, passive: true }`).

87. **Q: What is event bubbling and event capturing?** **A:** These are two phases of event propagation in the DOM:

- **Capturing Phase:** When an event occurs on an element, the browser first checks if any ancestors have registered a capturing listener for that event type, starting from the **window** down to the target element's parent.
- **Target Phase:** The event reaches the target element where it originated.
- **Bubbling Phase:** After the target phase, the event “bubbles” up through the target's ancestors, from the parent up to the **window**. Listeners registered for bubbling (default) will be triggered. You can control which phase your listener activates in using the third argument of `addEventListener`. `true` for capturing, `false` (or omitted) for bubbling.

88. **Q: What is `event.preventDefault()` and `event.stopPropagation()`?**

A:

- **event.preventDefault():** If an event is cancelable (e.g., a form

submission, a link click), calling this method prevents the browser's default action associated with that event.

- **event.stopPropagation():** Prevents further propagation of the current event in the capturing and bubbling phases. The event will not trigger listeners on any other elements further up or down the DOM tree.

89. **Q: What is localStorage and sessionStorage? What are their differences? A:** Both are Web Storage APIs that allow websites to store key-value pairs locally within the user's browser.

- **localStorage:**
 - Persists data even after the browser window is closed and re-opened.
 - Data is specific to the protocol/host/port (origin).
 - Storage limit is typically around 5-10MB.
 - Data is never sent to the server automatically.
- **sessionStorage:**
 - Persists data only for the duration of the page session (while the browser tab is open). Data is cleared when the tab/window is closed.
 - Data is specific to the origin and also isolated per browser tab/window.
 - Storage limit is similar to localStorage.
 - Data is never sent to the server automatically. Both store data as strings. You need `JSON.stringify()` and `JSON.parse()` for objects.

90. **Q: What are cookies? How do they differ from Web Storage? A:**

- **Cookies:** Small pieces of data stored by the browser for a specific domain. They are sent with every HTTP request to that domain. Differences from Web Storage (**localStorage/sessionStorage**):
- **Purpose:** Cookies are primarily for server-side communication (session management, tracking), while Web Storage is for client-side storage.
- **Data Transfer:** Cookies are sent with every HTTP request, potentially increasing request size. Web Storage data is not automatically sent.
- **Capacity:** Cookies have a smaller capacity (around 4KB). Web Storage has more (5-10MB).
- **Accessibility:** Cookies can be accessed on both server-side and client-side (if not `HttpOnly`). Web Storage is client-side only.
- **Expiration:** Cookies can have an expiration date. **localStorage** persists indefinitely, **sessionStorage** for the session.

VII. Error Handling & Miscellaneous (91-100)

91. Q: How do you handle errors in JavaScript? A:

- **try...catch...finally Statement:**
 - **try:** Code block to attempt execution.
 - **catch (error):** Code block to execute if an error occurs in the try block. The **error** object contains information about the error.
 - **finally:** Code block that executes regardless of whether an error occurred or not. Useful for cleanup.
- **throw Statement:** Manually throws a custom error. You can throw any expression (string, number, boolean, object). It's best to throw **Error** objects or instances of **Error** subclasses.
- **Window onerror Handler (Browser):** A global error handler for uncaught exceptions in the browser.
- **Node.js process.on('uncaughtException'):** Global error handler for uncaught exceptions in Node.js.
- **Promise .catch() and async/await try/catch:** For handling asynchronous errors.

92. Q: What are common types of errors in JavaScript? A:

- **SyntaxError:** Occurs when the JavaScript engine encounters code that violates the language's syntax rules (e.g., missing parenthesis, invalid keyword usage). Caught during parsing, before execution.
- **ReferenceError:** Occurs when trying to access a variable that has not been declared or is outside the current scope.
- **TypeError:** Occurs when an operation is performed on a value of an inappropriate type (e.g., calling a non-function, accessing properties of **null** or **undefined**).
- **RangeError:** Occurs when a numeric variable or parameter is outside its valid range (e.g., invalid array length).
- **URIError:** Occurs when global URI handling functions (like **encodeURIComponent()**) are used incorrectly.
- Custom errors can also be created by extending the **Error** object.

93. Q: What is “strict mode” (`'use strict';`) and what are some of its benefits? A: `'use strict';` is a directive that enables strict mode for JavaScript code. Benefits:

- **Changes “bad syntax” into real errors:** For example, assigning to a non-writable property, deleting an undeletable property, or using duplicate parameter names now throws errors.
- **Prevents accidental globals:** Assigning a value to an undeclared variable throws a **ReferenceError** instead of creating a global variable.
- **Makes eval() and arguments safer and less weird.**

- **this** is undefined in functions called without context (not methods), instead of the global object.
 - Prevents use of reserved keywords like **implements**, **interface**, **let**, **package**, **private**, **protected**, **public**, **static**, and **yield** as variable names.
 - Generally makes code more secure, robust, and potentially optimizable.
94. **Q: What are JavaScript Modules (ES6 Modules)? (import/export)**
A: ES6 Modules allow you to break your code into separate files (modules) for better organization, reusability, and maintainability.
- **export:** Used to make variables, functions, or classes available from a module to other modules.
 - **Named exports:** `export const myVar = ...; export function myFunc() {...}`
 - **Default export:** `export default function myFunc() {...}` (only one per module)
 - **import:** Used to bring exported members from another module into the current module's scope.
 - **Named imports:** `import { myVar, myFunc } from './myModule.js';`
 - **Default import:** `import myDefaultFunc from './myModule.js';`
 - **Namespace import:** `import * as MyModule from './myModule.js';`
 Modules are loaded asynchronously in browsers (if using `<script type="module">`) and have their own scope.
95. **Q: What is tree shaking?** **A:** Tree shaking is a term commonly used in the context of JavaScript module bundlers (like Webpack, Rollup, Parcel). It's a dead code elimination process. It works by analyzing **import** and **export** statements to detect which parts of your code (or imported libraries) are actually being used. Unused code is then “shaken off” from the final bundle, resulting in smaller bundle sizes and faster load times. This is particularly effective with ES6 modules due to their static structure.
96. **Q: What is the Symbol data type? Why might you use it?** **A:** **Symbol** (ES6) is a primitive data type whose instances are unique and immutable. Every **Symbol()** call returns a new, unique **Symbol**. **Use Cases:**
- **Unique Object Property Keys:** To create “hidden” or non-enumerable properties on objects that won't collide with string keys (e.g., for metadata or internal properties).
 - **Well-known Symbols:** JavaScript defines several built-in Symbols (e.g., **Symbol.iterator**, **Symbol.hasInstance**) that allow you to customize certain built-in language behaviors for your objects.
97. **Q: What is memoization? Provide a simple example.** **A:** Memoiza-

tion is an optimization technique where the results of expensive function calls are cached (stored) and returned when the same inputs occur again, avoiding redundant computation.

```
function memoize(fn) {
  const cache = {};
  return function(...args) {
    const key = JSON.stringify(args); // Create a unique key from arguments
    if (cache[key]) {
      console.log("Fetching from cache for key:", key);
      return cache[key];
    } else {
      console.log("Calculating result for key:", key);
      const result = fn(...args);
      cache[key] = result;
      return result;
    }
  };
}

function slowAdd(a, b) {
  // Simulate a slow operation
  for (let i = 0; i < 1e8; i++) {}
  return a + b;
}

const memoizedAdd = memoize(slowAdd);
console.log(memoizedAdd(2, 3)); // Calculating result for key: [2,3] -> 5
console.log(memoizedAdd(2, 3)); // Fetching from cache for key: [2,3] -> 5
console.log(memoizedAdd(5, 5)); // Calculating result for key: [5,5] -> 10
```

98. **Q: What is the console object and some of its useful methods beyond log()?** **A:** The console object provides access to the browser's debugging console or Node.js console. Useful methods:

- `console.log()`: General output of logging information.
- `console.error()`: Outputs an error message. Often styled differently.
- `console.warn()`: Outputs a warning message.
- `console.info()`: Outputs an informational message.
- `console.table(data)`: Displays tabular data as a table.
- `console.dir(object)`: Displays an interactive list of the properties of a JavaScript object.
- `console.group(label)` / `console.groupEnd()`: Creates a new inline group, indenting all following output. Call `groupEnd` to exit the group.
- `console.groupCollapsed(label)`: Similar to `group` but starts col-

lapsed.

- `console.time(label)` / `console.timeEnd(label)`: Starts a timer to track how long an operation takes.
- `console.trace()`: Outputs a stack trace.
- `console.assert(assertion, message)`: Logs a message and stack trace if an assertion is false.
- `console.clear()`: Clears the console.

99. **Q: What are some common JavaScript performance optimization techniques? A:**

- **Minimize DOM Manipulation:** DOM operations are expensive. Batch updates, use DocumentFragments, or update elements off-DOM.
- **Debouncing and Throttling:** For event handlers that fire frequently (e.g., scroll, resize, keyup).
 - *Debounce*: Delays function execution until a certain amount of time has passed without the event firing.
 - *Throttle*: Ensures a function is called at most once per specified interval.
- **Efficient Looping:** Use appropriate loops. `for` loops are often faster than `forEach` for very large arrays in performance-critical sections.
- **Code Splitting / Lazy Loading:** Load only the JavaScript needed for the current view/functionality.
- **Tree Shaking / Dead Code Elimination:** Remove unused code.
- **Caching / Memoization:** Store results of expensive computations.
- **Optimize Images and Assets:** Compress images, use appropriate formats.
- **Use Web Workers for CPU-intensive tasks.**
- **Avoid Global Variables:** Faster lookup for local variables.
- **Minimize Reflows and Repaints:** Changing styles or layout can trigger these.
- **Use efficient data structures (e.g., Map, Set when appropriate).**

100. **Q: If you encountered a bug in production JavaScript code, what would be your general approach to debugging it? A:** 1. **Reproduce the Bug:** Try to reliably reproduce the issue in a development or staging environment. Understand the steps that lead to it. 2. **Gather Information:**

- Check browser console for error messages, stack traces, and logs.
- Note the browser version, OS, and any specific user actions.
- If server-side (Node.js), check server logs. 3. **Isolate the Problem:**
- Use `console.log` statements or the debugger to trace variable values and execution flow around the suspected area.

- Comment out sections of code to narrow down where the error originates.
 - If using version control (like Git), check recent changes (`git blame`, `git bisect`) that might have introduced the bug.
4. **Use Browser Developer Tools:**
- **Debugger:** Set breakpoints, step through code, inspect variables, watch expressions.
 - **Network Tab:** Inspect API requests and responses if it's a data-related bug.
 - **Elements Tab:** Inspect DOM structure and styles if it's a UI bug.
5. **Formulate a Hypothesis:** Based on the information, make an educated guess about the cause.
6. **Test the Hypothesis:** Try a fix. If it doesn't work, re-evaluate.
7. **Consider Edge Cases:** Once a potential fix is found, think about related edge cases or scenarios it might affect.
8. **Write a Test (if applicable):** For complex bugs, write a unit or integration test that reproduces the bug to ensure the fix works and to prevent regressions.
9. **Deploy Fix & Monitor:** Deploy the corrected code and monitor to ensure the bug is resolved and no new issues are introduced.
10. **Documentation/Root Cause Analysis:** Document the bug, its cause, and the solution for future reference, especially for complex or recurring issues.

Phew! That's 100. I hope this comprehensive list is helpful!